Project Report

Abstract
This project implements a blockchain-based system to register and fractionalize the African Leadership University (ALU) logo. By leveraging an ERC-721 smart contract to establish immutable proof of authenticity and an ERC-20 contract to split ownership into distributable tokens, this solution ensures transparent, decentralized verification and stakeholder ownership tracking.

Introduction

ALU needs to protect its logo on a blockchain because traditional databases and copyright registries rely on centralized authorities, which are vulnerable to alteration, deletion, or loss. A blockchain provides an immutable, decentralized ledger where the exact state and timestamp of registration are permanently recorded. This ensures undeniable cryptographic proof of existence and authenticity that any party can verify independently without relying on a central entity.

Blockchain Setup

A local blockchain is a simulated network running locally on a developer's computer. Developers use it instead of the real Ethereum network to test smart contracts for free, without spending real ETH, and with instant transaction mining.

Hardhat was set up to spin up this local node. It compiles the Solidity contracts, creates a local blockchain environment, and provides dummy wallet addresses pre-funded with fake ETH. A wallet address is a unique identifier (like a bank account number) on the blockchain that represents a user's account, holding their funds and interacting with smart contracts.

Smart Contract Explanation

A smart contract is a self-executing program deployed on a blockchain. Unlike standard programs that run on centralized servers (where code can be secretly changed by administrators), smart contracts run on a decentralized network where their code and data are public, immutable, and strictly enforce the agreed-upon logic.

An NFT (Non-Fungible Token) is a unique digital asset. The ERC-721 standard is the industry standard for NFTs because it allows each token to have distinct properties and a unique ID, making it ideal for registering a one-of-a-kind asset like the ALU Logo.

A SHA-256 hash is a cryptographic function that converts any file into a unique 64-character string of letters and numbers. Changing even a single pixel in an image alters the file's binary data completely, resulting in a fundamentally different hash (the "avalanche effect").

Data stored on a blockchain cannot be changed because each block of data is cryptographically linked to the previous one in a chain. Altering data requires rewriting the entire chain and gaining majority control over a massive decentralized network, making it practically impossible.

Functions Created:
• registerAsset(): Evaluates the inputted name, file type, and file hash. Rejects duplicate hashes, issues an NFT, and links metadata to the token ID.
• verifyLogoIntegrity(): Verifies whether a submitted hash matches the stored hash.
• getAsset(): Returns the asset registration details.
• distributeShares(): Sends ALUT ownership tokens to a specified wallet.
• ownershipPercentage(): Returns the percentage of total ownership held by a wallet.

ALU Logo Hash: Replace this with the hash generated when running `npx hardhat run scripts/deploy.js`.

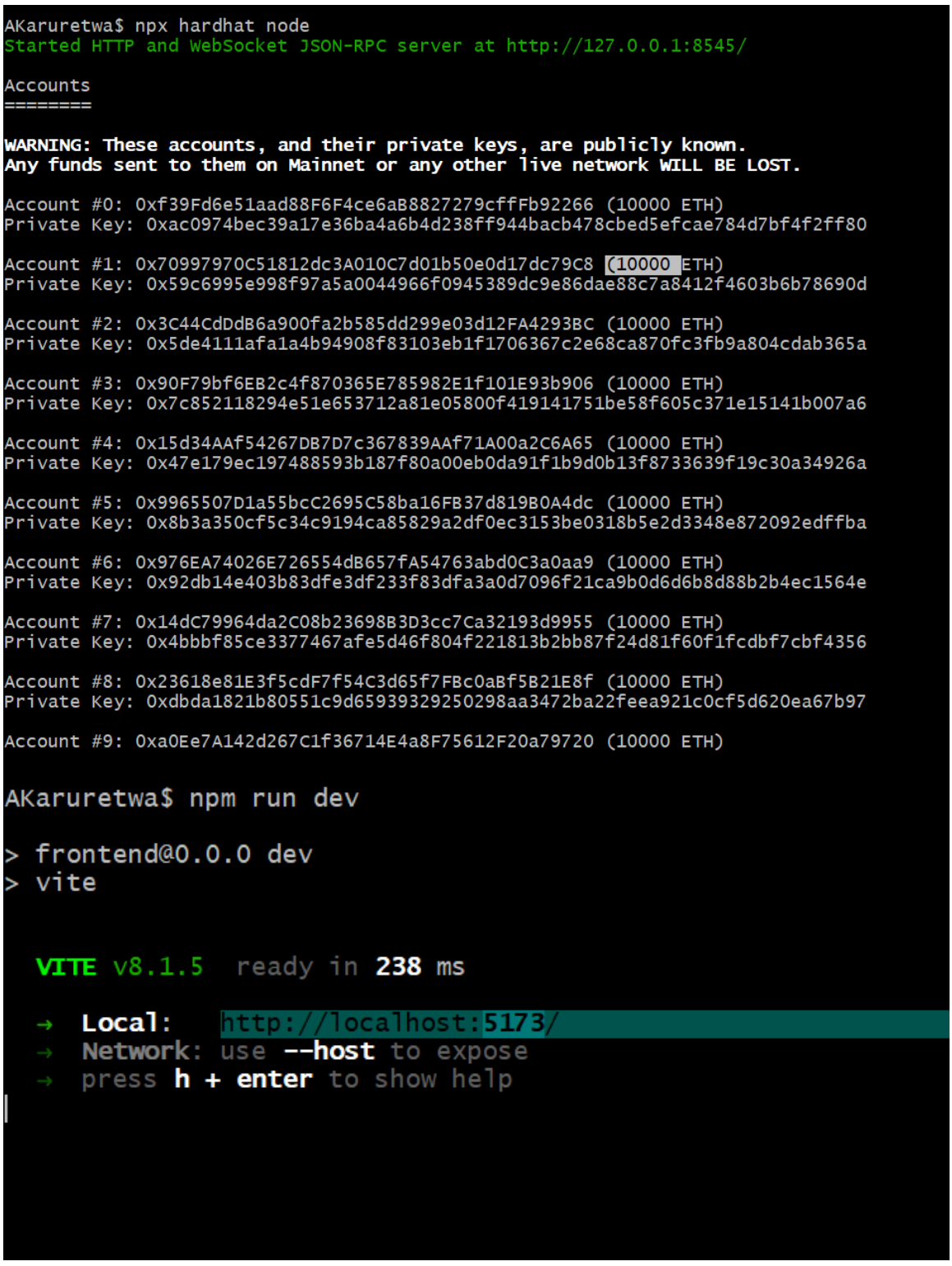
Tokenisation
Tokenisation is the process of splitting ownership of a single asset into multiple digital tokens, allowing multiple people to hold fractional stakes.

For the ALU logo, tokens could be distributed to the university founder, the primary designer, and student council members or alumni.

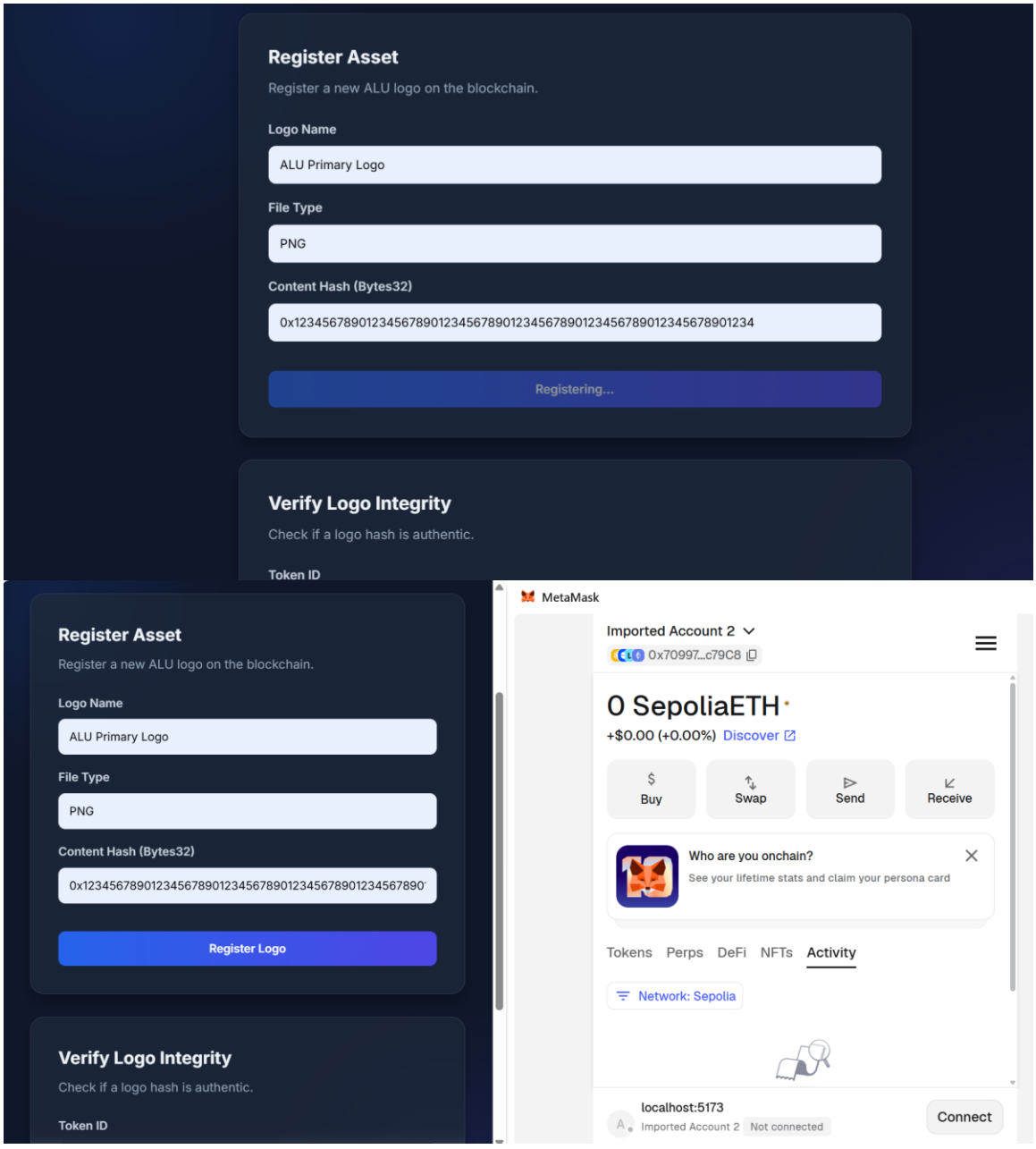
ERC-721 is used because the logo registration is unique. ERC-20 is used because ownership shares are fungible.

If a faculty member holds 100,000 ALUT tokens out of 1,000,000 total, they own a verifiable 10% share of the asset.

Test Results



```
AKaruretwa$ npx hardhat console --network localhost
Welcome to Node.js v24.14.1.
Type ".help" for more information.
> await ethers.getSigners()
[
  HardhatEthersSigner {
    _gasLimit: 16777216,
    address: '0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266',
    provider: HardhatEthersProvider {
      _hardhatProvider: [LazyInitializationProviderAdapter],
      _networkName: 'localhost',
      _blockListeners: [],
      _transactionHashListeners: Map(0) {},
      _eventListeners: []
    },
    _accounts: 'remote'
  },
  HardhatEthersSigner {
    _gasLimit: 16777216,
    address: '0x70997970C51812dc3A010C7d01b50e0d17dc79C8',
    provider: HardhatEthersProvider {
      _hardhatProvider: [LazyInitializationProviderAdapter],
      _networkName: 'localhost',
      _blockListeners: [],
      _transactionHashListeners: Map(0) {},
      _eventListeners: []
    },
    _accounts: 'remote'
  },
  HardhatEthersSigner {
    _gasLimit: 16777216,
    address: '0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC',
    provider: HardhatEthersProvider {
      _hardhatProvider: [LazyInitializationProviderAdapter],
      _networkName: 'localhost',
      _blockListeners: [],
      _transactionHashListeners: Map(0) {},
      _eventListeners: []
    },
    _accounts: 'remote'
  },
  HardhatEthersSigner {
    _gasLimit: 16777216,
    address: '0x90F79bf6EB2c4f870365E785982E1f101E93b906',
    provider: HardhatEthersProvider {
      _hardhatProvider: [LazyInitializationProviderAdapter],
      _networkName: 'localhost',
      _blockListeners: [],
      _transactionHashListeners: Map(0) {},
      _eventListeners: []
    },
    _accounts: 'remote'
```



Conclusion

Through this project, I learned how to combine ERC-721 and ERC-20 standards to register and tokenize a digital asset. With more time, I would build a React front-end for uploading images, hashing files, and distributing ownership shares.

GitHub Link

https://github.com/TWA-Coder/alu/tree/main/alu-logo-blockchain

Video Link

https://www.awesomescreenshot.com/s/folder/F0PJpawyq8/a497fba71693471f295b84b72c5c87e7